

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Newton's Equations of Motion</title>
<style>
*, ::before, ::after {
box-sizing: border-box;
margin: 0;
padding: 0;
user-select: none;
}

body {
background-color: #03070d;
color: #e6f6ff;
font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", sans-serif;
overflow-x: hidden;
height: 800vh; /* Scroll runway */
scrollbar-width: thin;
scrollbar-color: #00d2ff #03070d;
}

::-webkit-scrollbar {
width: 6px;
}
::-webkit-scrollbar-track {
background: #03070d;
}
::-webkit-scrollbar-thumb {
background: #00d2ff;
border-radius: 3px;
box-shadow: 0 0 10px #00d2ff;
}

/* Fixed Stage */
#viewport {
position: fixed;
top: 0;
left: 0;
width: 100vw;
height: 100vh;
display: flex;
flex-direction: column;
justify-content: space-between;
pointer-events: none;
z-index: 10;
}

/* Deep space glowing grid background */
canvas#bg-canvas {
position: fixed;
top: 0;
left: 0;
width: 100vw;
height: 100vh;
```

```
z-index: 1;
pointer-events: none;
}
```

```
/* Physics View Canvas */
canvas#physics-canvas {
position: fixed;
top: 0;
left: 0;
width: 100vw;
height: 100vh;
z-index: 2;
pointer-events: none;
}
```

```
/* Futuristic UI Elements */
header {
padding: 32px 48px;
display: flex;
justify-content: space-between;
align-items: flex-start;
}
```

```
.title-block {
display: flex;
flex-direction: column;
gap: 4px;
}
```

```
.title-block h1 {
font-size: 1.15rem;
letter-spacing: 0.25em;
text-transform: uppercase;
color: #ffffff;
font-weight: 700;
text-shadow: 0 0 12px rgba(0, 210, 255, 0.8), 0 0 30px rgba(0, 210, 255, 0.4);
}
```

```
.title-block span {
font-size: 0.65rem;
letter-spacing: 0.2em;
color: rgba(200, 235, 255, 0.4);
text-transform: uppercase;
}
```

```
.scroll-hint {
display: flex;
align-items: center;
gap: 10px;
font-size: 0.7rem;
letter-spacing: 0.25em;
text-transform: uppercase;
color: #00d2ff;
opacity: 0.8;
}
```

```
.pulse-dot {
width: 7px;
```

```
height: 7px;
background: #00d2ff;
border-radius: 50%;
box-shadow: 0 0 10px #00d2ff, 0 0 20px #00d2ff;
animation: pulse 1.6s ease-in-out infinite alternate;
}
```

```
@keyframes pulse {
0% { transform: scale(0.8); opacity: 0.3; }
100% { transform: scale(1.4); opacity: 1; }
}
```

```
/* Active Stage Center Overlay */
```

```
.stage-overlay {
position: absolute;
top: 18%;
left: 50%;
transform: translateX(-50%);
display: flex;
flex-direction: column;
align-items: center;
gap: 10px;
pointer-events: none;
text-align: center;
width: 90%;
max-width: 800px;
}
```

```
.stage-badge {
font-size: 0.65rem;
letter-spacing: 0.3em;
text-transform: uppercase;
padding: 4px 12px;
border-radius: 100px;
border: 1px solid rgba(0, 210, 255, 0.3);
background: rgba(0, 210, 255, 0.05);
color: #00d2ff;
backdrop-filter: blur(8px);
}
```

```
.equation-display {
font-size: clamp(1.8rem, 4vw, 3.2rem);
font-weight: 300;
letter-spacing: 0.06em;
color: #ffffff;
text-shadow: 0 0 16px rgba(0, 210, 255, 0.9), 0 0 40px rgba(0, 150, 255, 0.4);
transition: all 0.3s ease;
}
```

```
.equation-display b {
font-weight: 700;
color: #00d2ff;
}
```

```
/* Real-time Telemetry Dashboard Bottom */
```

```
.telemetry-bar {
padding: 24px 48px;
display: flex;
```

```
justify-content: space-between;
align-items: flex-end;
width: 100%;
background: linear-gradient(180deg, transparent 0%, rgba(3, 7, 13, 0.95) 80%);
}
```

```
.telemetry-grid {
display: flex;
gap: 32px;
}
```

```
.tele-item {
display: flex;
flex-direction: column;
gap: 2px;
}
```

```
.tele-label {
font-size: 0.62rem;
letter-spacing: 0.2em;
text-transform: uppercase;
color: rgba(255, 255, 255, 0.4);
}
```

```
.tele-val-box {
display: flex;
align-items: baseline;
gap: 4px;
}
```

```
.tele-val {
font-size: 1.4rem;
font-weight: 600;
font-family: monospace;
color: #ffffff;
text-shadow: 0 0 10px rgba(0, 210, 255, 0.7);
}
```

```
.tele-unit {
font-size: 0.7rem;
color: #00d2ff;
}
```

```
.progress-bar-container {
width: 220px;
display: flex;
flex-direction: column;
gap: 6px;
}
```

```
.progress-track {
width: 100%;
height: 3px;
background: rgba(255, 255, 255, 0.08);
border-radius: 2px;
overflow: hidden;
}
```

```
.progress-fill {
height: 100%;
width: 0%;
background: #00d2ff;
box-shadow: 0 0 10px #00d2ff;
}
```

```
.progress-label {
display: flex;
justify-content: space-between;
font-size: 0.6rem;
letter-spacing: 0.15em;
color: rgba(255, 255, 255, 0.4);
}
```

```
@media (max-width: 768px) {
header { padding: 20px 24px; }
.telemetry-bar { padding: 16px 24px; flex-direction: column; gap: 16px; align-items: stretch; }
.telemetry-grid { justify-content: space-between; gap: 8px; }
.progress-bar-container { width: 100%; }
.tele-val { font-size: 1.1rem; }
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<canvas id="bg-canvas"></canvas>
```

```
<canvas id="physics-canvas"></canvas>
```

```
<div id="viewport">
```

```
<header>
```

```
<div class="title-block">
```

```
<h1>Equations of Motion</h1>
```

```
<span>Kinematics Visualizer</span>
```

```
</div>
```

```
<div class="scroll-hint">
```

```
<span class="pulse-dot"></span>
```

```
<span>Scroll to Simulate</span>
```

```
</div>
```

```
</header>
```

```
<div class="stage-overlay">
```

```
<div class="stage-badge" id="stage-badge">PHASE 01 // DISPLACEMENT</div>
```

```
<div class="equation-display" id="equation-display"> $s = \int v \, dt$ </div>
```

```
</div>
```

```
<div class="telemetry-bar">
```

```
<div class="telemetry-grid">
```

```
<div class="tele-item">
```

```
<div class="tele-label">Initial (u)</div>
```

```
<div class="tele-val-box">
```

```
<span class="tele-val" id="val-u">0.00</span>
```

```
<span class="tele-unit">m/s</span>
```

```
</div>
```

```
</div>
```

```
<div class="tele-item">
```

```
<div class="tele-label">Velocity (v)</div>
```

```
<div class="tele-val-box">
```

```

<span class="tele-val" id="val-v">0.00</span>
<span class="tele-unit">m/s</span>
</div>
</div>
<div class="tele-item">
<div class="tele-label">Accel (a)</div>
<div class="tele-val-box">
<span class="tele-val" id="val-a">0.00</span>
<span class="tele-unit">m/s2</span>
</div>
</div>
<div class="tele-item">
<div class="tele-label">Time (t)</div>
<div class="tele-val-box">
<span class="tele-val" id="val-t">0.00</span>
<span class="tele-unit">s</span>
</div>
</div>
<div class="tele-item">
<div class="tele-label">Displacement (s)</div>
<div class="tele-val-box">
<span class="tele-val" id="val-s">0.00</span>
<span class="tele-unit">m</span>
</div>
</div>
</div>

```

```

<div class="progress-bar-container">
<div class="progress-label">
<span>TIMELINE</span>
<span id="pct-label">0%</span>
</div>
<div class="progress-track">
<div class="progress-fill" id="progress-fill"></div>
</div>
</div>
</div>
</div>

```

```

<script>
const bgCanvas = document.getElementById('bg-canvas');
const physCanvas = document.getElementById('physics-canvas');
const bgCtx = bgCanvas.getContext('2d');
const ctx = physCanvas.getContext('2d');

```

```

// UI Elements
const stageBadge = document.getElementById('stage-badge');
const eqDisplay = document.getElementById('equation-display');
const valU = document.getElementById('val-u');
const valV = document.getElementById('val-v');
const valA = document.getElementById('val-a');
const valT = document.getElementById('val-t');
const valS = document.getElementById('val-s');
const progressFill = document.getElementById('progress-fill');
const pctLabel = document.getElementById('pct-label');

```

```

let width, height;
let scrollProgress = 0;

```

```
let smoothProgress = 0;
```

```
// History buffer for velocity vs time miniature real-time graph  
let trailHistory = [];
```

```
function resize() {  
  width = window.innerWidth;  
  height = window.innerHeight;  
  bgCanvas.width = width * window.devicePixelRatio;  
  bgCanvas.height = height * window.devicePixelRatio;  
  physCanvas.width = width * window.devicePixelRatio;  
  physCanvas.height = height * window.devicePixelRatio;  
  bgCanvas.style.width = width + 'px';  
  bgCanvas.style.height = height + 'px';  
  physCanvas.style.width = width + 'px';  
  physCanvas.style.height = height + 'px';  
  bgCtx.scale(window.devicePixelRatio, window.devicePixelRatio);  
  ctx.scale(window.devicePixelRatio, window.devicePixelRatio);  
  drawGrid();  
}
```

```
window.addEventListener('resize', resize);
```

```
// Background Grid Drawing (High-tech graph paper)
```

```
function drawGrid() {  
  bgCtx.clearRect(0, 0, width, height);  
  bgCtx.lineWidth = 0.5;  
  const step = 45;
```

```
// Sub-grid lines
```

```
bgCtx.strokeStyle = 'rgba(0, 180, 255, 0.035)';  
for (let x = 0; x < width; x += step) {  
  bgCtx.beginPath();  
  bgCtx.moveTo(x, 0);  
  bgCtx.lineTo(x, height);  
  bgCtx.stroke();  
}  
for (let y = 0; y < height; y += step) {  
  bgCtx.beginPath();  
  bgCtx.moveTo(0, y);  
  bgCtx.lineTo(width, y);  
  bgCtx.stroke();  
}
```

```
// Radial center vignette
```

```
const grad = bgCtx.createRadialGradient(width / 2, height / 2, 100, width / 2, height / 2, Math.max(width, height) * 0.7);  
grad.addColorStop(0, 'rgba(0, 30, 60, 0)');  
grad.addColorStop(1, 'rgba(2, 6, 12, 0.95)');  
bgCtx.fillStyle = grad;  
bgCtx.fillRect(0, 0, width, height);  
}
```

```
// Smooth scroll interpolation
```

```
window.addEventListener('scroll', () => {  
  const maxScroll = document.documentElement.scrollHeight - window.innerHeight;  
  scrollProgress = Math.min(Math.max(window.scrollY / maxScroll, 0), 1);  
});
```

```

// Phases and Kinematic States
function computeState(p) {
  let u = 0, v = 0, a = 0, t = 0, s = 0;
  let badge = "";
  let equation = "";

  if (p < 0.25) {
    // PHASE 1: DISPLACEMENT (Constant Velocity:  $a = 0, s = v \cdot t$ )
    const localP = p / 0.25;
    badge = "PHASE 01 // DISPLACEMENT & TIME";
    equation = " $s = u \cdot t$ ";
    u = 24.0;
    a = 0.0;
    t = localP * 6.0;
    v = u;
    s = u * t;
  } else if (p < 0.50) {
    // PHASE 2: 1st EQUATION OF MOTION ( $v = u + at$ )
    const localP = (p - 0.25) / 0.25;
    badge = "PHASE 02 // 1ST EQUATION OF MOTION";
    equation = " $v = u + a \cdot t$ ";
    u = 12.0;
    a = 8.0;
    t = localP * 5.0;
    v = u + (a * t);
    s = (u * t) + (0.5 * a * t * t);
  } else if (p < 0.75) {
    // PHASE 3: 2nd EQUATION OF MOTION ( $s = ut + \frac{1}{2} a t^2$ )
    const localP = (p - 0.50) / 0.25;
    badge = "PHASE 03 // 2ND EQUATION OF MOTION";
    equation = " $s = u \cdot t + \frac{1}{2} \cdot a \cdot t^2$ ";
    u = 5.0;
    a = 15.0;
    t = localP * 4.5;
    v = u + (a * t);
    s = (u * t) + (0.5 * a * t * t);
  } else {
    // PHASE 4: 3rd EQUATION OF MOTION ( $v^2 = u^2 + 2as$ )
    const localP = (p - 0.75) / 0.25;
    badge = "PHASE 04 // 3RD EQUATION OF MOTION";
    equation = " $v^2 = u^2 + 2 \cdot a \cdot s$ ";
    u = 8.0;
    a = 18.0;
    t = localP * 4.0;
    v = Math.sqrt(Math.max(0, (u * u) + (2 * a * (0.5 * a * t * t + u * t))));
    s = (u * t) + (0.5 * a * t * t);
  }

  return { u, v, a, t, s, badge, equation };
}

```

```

// Draw arrow with glowing head & label
function drawVectorArrow(startX, startY, length, color, label, valueText, glowColor) {
  if (Math.abs(length) < 2) return;

  ctx.save();
  ctx.strokeStyle = color;

```

```

ctx.fillStyle = color;
ctx.lineWidth = 2.5;
ctx.shadowColor = glowColor || color;
ctx.shadowBlur = 12;

const endX = startX + length;
const endY = startY;
const dir = Math.sign(length);

// Stem
ctx.beginPath();
ctx.moveTo(startX, startY);
ctx.lineTo(endX, endY);
ctx.stroke();

// Arrowhead
const headLen = 10 * dir;
ctx.beginPath();
ctx.moveTo(endX, endY);
ctx.lineTo(endX - headLen, endY - 5);
ctx.lineTo(endX - headLen, endY + 5);
ctx.closePath();
ctx.fill();

// Label
ctx.font = '600 10px monospace';
ctx.letterSpacing = "0.08em";
ctx.fillText(`${label} ${valueText}`, startX + length * 0.5 - 20, startY - 10);
ctx.restore();
}

// Render Sci-Fi Aerodynamic Particle Capsule (Vehicle)
function drawVehicle(x, y, v, a) {
  ctx.save();
  ctx.translate(x, y);

  // Glowing Trail
  const tailGrad = ctx.createLinearGradient(-70, 0, 0, 0);
  tailGrad.addColorStop(0, 'rgba(0, 210, 255, 0)');
  tailGrad.addColorStop(1, 'rgba(0, 210, 255, 0.45)');
  ctx.fillStyle = tailGrad;
  ctx.beginPath();
  ctx.moveTo(-60 - Math.min(v * 0.9, 120), -4);
  ctx.lineTo(0, -9);
  ctx.lineTo(0, 9);
  ctx.lineTo(-60 - Math.min(v * 0.9, 120), 4);
  ctx.closePath();
  ctx.fill();

  // Outer Aura
  ctx.shadowColor = '#00e1ff';
  ctx.shadowBlur = 24;

  // Body Hull
  ctx.strokeStyle = '#e6faff';
  ctx.fillStyle = '#061626';
  ctx.lineWidth = 2;

```

```
ctx.beginPath();
ctx.moveTo(35, 0);
ctx.lineTo(15, -12);
ctx.lineTo(-30, -12);
ctx.lineTo(-40, -4);
ctx.lineTo(-40, 4);
ctx.lineTo(-30, 12);
ctx.lineTo(15, 12);
ctx.closePath();
ctx.fill();
ctx.stroke();
```

```
// Neon Accents
ctx.strokeStyle = '#00f2ff';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.moveTo(-20, -5);
ctx.lineTo(15, -5);
ctx.stroke();
```

```
ctx.beginPath();
ctx.moveTo(-20, 5);
ctx.lineTo(15, 5);
ctx.stroke();
```

```
// Core Reactor / Engine Core
ctx.fillStyle = 'ffffff';
ctx.beginPath();
ctx.arc(-2, 0, 4.5, 0, Math.PI * 2);
ctx.fill();
```

```
// Wheels / Thruster Nodes
ctx.strokeStyle = '#00c3ff';
ctx.fillStyle = '#03080e';
ctx.lineWidth = 2;
[-22, 18].forEach(pos => {
  ctx.beginPath();
  ctx.arc(pos, 14, 5, 0, Math.PI * 2);
  ctx.fill();
  ctx.stroke();
```

```
ctx.beginPath();
ctx.arc(pos, -14, 5, 0, Math.PI * 2);
ctx.fill();
ctx.stroke();
});
```

```
ctx.restore();
}
```

```
// Draw Real-time Miniature Telemetry Graph (Velocity-Time Curve)
function drawMiniGraph(originX, originY, graphW, graphH, currentT, currentV) {
  ctx.save();
  ctx.translate(originX, originY);
```

```
// Frame
ctx.strokeStyle = 'rgba(0, 210, 255, 0.2)';
ctx.fillStyle = 'rgba(2, 10, 20, 0.6)';
```

```

ctx.lineWidth = 1;
ctx.fillRect(0, 0, graphW, graphH);
ctx.strokeRect(0, 0, graphW, graphH);

// Grid lines inside graph
ctx.strokeStyle = 'rgba(0, 210, 255, 0.08)';
for (let gx = 0; gx < graphW; gx += 25) {
  ctx.beginPath();
  ctx.moveTo(gx, 0); ctx.lineTo(gx, graphH); ctx.stroke();
}
for (let gy = 0; gy < graphH; gy += 20) {
  ctx.beginPath();
  ctx.moveTo(0, gy); ctx.lineTo(graphW, gy); ctx.stroke();
}

// Axes labels
ctx.fillStyle = 'rgba(200, 235, 255, 0.5)';
ctx.font = '8px monospace';
ctx.fillText('v(t)', 6, 12);
ctx.fillText('t →', graphW - 20, graphH - 5);

// Plot curve history
if (trailHistory.length > 1) {
  ctx.beginPath();
  ctx.strokeStyle = '#00f2ff';
  ctx.shadowColor = '#00f2ff';
  ctx.shadowBlur = 8;
  ctx.lineWidth = 1.8;

  for (let i = 0; i < trailHistory.length; i++) {
    const hx = (trailHistory[i].t / 6.0) * (graphW - 16) + 8;
    const hy = graphH - ((trailHistory[i].v / 90.0) * (graphH - 20) + 8);
    if (i === 0) ctx.moveTo(hx, hy);
    else ctx.lineTo(hx, hy);
  }
  ctx.stroke();

  // Plot point on tip
  const latest = trailHistory[trailHistory.length - 1];
  const lx = (latest.t / 6.0) * (graphW - 16) + 8;
  const ly = graphH - ((latest.v / 90.0) * (graphH - 20) + 8);
  ctx.fillStyle = 'ffffff';
  ctx.beginPath();
  ctx.arc(lx, ly, 3, 0, Math.PI * 2);
  ctx.fill();
}

ctx.restore();
}

// Animation Loop
let lastTimestamp = 0;
function animate(timestamp) {
  // Smooth interpolation (lerp)
  smoothProgress += (scrollProgress - smoothProgress) * 0.1;

  const state = computeState(smoothProgress);

```

```

// Update UI numbers
valU.innerText = state.u.toFixed(2);
valV.innerText = state.v.toFixed(2);
valA.innerText = state.a.toFixed(2);
valT.innerText = state.t.toFixed(2);
valS.innerText = state.s.toFixed(2);
stageBadge.innerText = state.badge;
eqDisplay.innerHTML = state.equation;

const pct = Math.round(smoothProgress * 100);
progressFill.style.width = pct + '%';
pctLabel.innerText = pct + '%';

// Store sample history for mini-plot (max 60 samples)
if (trailHistory.length === 0 || Math.abs(trailHistory[trailHistory.length - 1].t - state.t) > 0.04) {
  trailHistory.push({ t: state.t, v: state.v });
  if (trailHistory.length > 50) trailHistory.shift();
}
// Clear history on phase wrap-around
if (trailHistory.length > 2 && trailHistory[trailHistory.length - 1].t < trailHistory[trailHistory.length - 2].t) {
  trailHistory = [];
}

// Canvas Clear
ctx.clearRect(0, 0, width, height);

// Primary Track Runway Geometry
const trackY = height * 0.58;
const trackStart = width * 0.08;
const trackEnd = width * 0.92;
const trackWidth = trackEnd - trackStart;

// Displacement Mapping
const maxTrackDisplacement = 280; // normalized max s in meters
const normalizedS = Math.min(state.s / maxTrackDisplacement, 1);
const carX = trackStart + normalizedS * trackWidth;

// Draw Main Linear Track
ctx.save();
ctx.strokeStyle = 'rgba(0, 210, 255, 0.2)';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.moveTo(trackStart, trackY);
ctx.lineTo(trackEnd, trackY);
ctx.stroke();

// Illuminated displacement trail on track
ctx.strokeStyle = '#00d2ff';
ctx.shadowColor = '#00d2ff';
ctx.shadowBlur = 12;
ctx.lineWidth = 3;
ctx.beginPath();
ctx.moveTo(trackStart, trackY);
ctx.lineTo(carX, trackY);
ctx.stroke();
ctx.restore();

// Track Distance Markers (Every 25 meters equivalent)

```

```

const markerCount = 11;
for (let i = 0; i < markerCount; i++) {
const mx = trackStart + (i / (markerCount - 1)) * trackWidth;
const distVal = Math.round((i / (markerCount - 1)) * maxTrackDisplacement);

ctx.save();
const isActive = mx <= carX;
ctx.strokeStyle = isActive ? '#00e1ff' : 'rgba(255, 255, 255, 0.2)';
ctx.fillStyle = isActive ? '#00e1ff' : 'rgba(255, 255, 255, 0.3)';
ctx.shadowColor = isActive ? '#00e1ff' : 'transparent';
ctx.shadowBlur = isActive ? 8 : 0;
ctx.lineWidth = 1;

// Tick mark
ctx.beginPath();
ctx.moveTo(mx, trackY - 8);
ctx.lineTo(mx, trackY + 8);
ctx.stroke();

// Distance Label
ctx.font = '9px monospace';
ctx.textAlign = 'center';
ctx.fillText(`${distVal}m`, mx, trackY + 24);
ctx.restore();
}

// Origin Marker (x = 0)
ctx.save();
ctx.strokeStyle = '#ffffff';
ctx.fillStyle = '#ffffff';
ctx.shadowColor = '#00e1ff';
ctx.shadowBlur = 10;
ctx.beginPath();
ctx.arc(trackStart, trackY, 3, 0, Math.PI * 2);
ctx.fill();
ctx.font = '10px monospace';
ctx.textAlign = 'center';
ctx.fillText("s = 0", trackStart, trackY - 18);
ctx.restore();

// Draw Dynamic Vectors from Car
// Velocity Vector Arrow (Greenish-cyan neon)
const velArrowLen = Math.min(state.v * 2.2, 180);
drawVectorArrow(carX, trackY - 32, velArrowLen, '#00ffcc', 'v', `${state.v.toFixed(1)} m/s`, '#00ffcc');

// Acceleration Vector Arrow (Deep Sky Neon Blue)
if (state.a > 0) {
const accArrowLen = Math.min(state.a * 6.5, 150);
drawVectorArrow(carX, trackY - 50, accArrowLen, '#0099ff', 'a', `${state.a.toFixed(1)} m/s²`, '#0099ff');
}

// Displacement Ribbon / Bracket from Start to Car
if (carX - trackStart > 30) {
ctx.save();
ctx.strokeStyle = 'rgba(0, 225, 255, 0.5)';
ctx.lineWidth = 1.5;
ctx.setLineDash([4, 4]);
const bracketY = trackY + 44;

```

```

ctx.beginPath();
ctx.moveTo(trackStart, bracketY);
ctx.lineTo(carX, bracketY);
ctx.stroke();

ctx.setLineDash([]);
// Start/End brackets
ctx.beginPath();
ctx.moveTo(trackStart, bracketY - 5); ctx.lineTo(trackStart, bracketY + 5);
ctx.moveTo(carX, bracketY - 5); ctx.lineTo(carX, bracketY + 5);
ctx.stroke();

// Displacement text in center
ctx.fillStyle = 'ffffff';
ctx.font = '10px monospace';
ctx.textAlign = 'center';
ctx.shadowColor = '#00d2ff';
ctx.shadowBlur = 8;
ctx.fillText(`Δs = ${state.s.toFixed(1)} m`, (trackStart + carX) / 2, bracketY + 16);
ctx.restore();
}

// Draw Real-Time Vehicle Entity
drawVehicle(carX, trackY, state.v, state.a);

// Draw Live Embedded Mini-Plot (Velocity vs Time) in top right area
const plotW = Math.min(220, width * 0.28);
const plotH = 100;
const plotX = width - plotW - 48;
const plotY = 95;
if (width > 600) {
drawMiniGraph(plotX, plotY, plotW, plotH, state.t, state.v);
}

requestAnimationFrame(animate);
}

// Initialization
resize();
requestAnimationFrame(animate);
</script>
</body>
</html>

```